Project Proposal: AquaPure AI

Smart Aquaculture & Automated Water Quality Management System

Student Name:	Siam Joadder Shoishob
Student ID:	IT23049
Course Title:	Software Engineering and Project Management
Date:	October 26, 2026

Executive Summary

AquaPure AI is an IoT-enabled, AI-driven Java web application designed for modern aquaculture management. By continuously monitoring key water parameters—such as Dissolved Oxygen (DO), pH, temperature, and turbidity—via submerged sensors, AquaPure AI predicts water quality deterioration using machine learning models. The system automatically triggers aeration and feeding actuators while sending real-time alerts to farm operators. This proposal outlines the system architecture, software process model, complete UML representations, and a working core Java interface implementation.

1. Introduction & Project Overview

Aquaculture plays a critical role in global food security. However, traditional fish farming heavily relies on manual water testing, which is prone to human error and delays. Sudden drops in Dissolved Oxygen (DO) levels can lead to catastrophic fish mortality within hours.

AquaPure AI solves this problem by integrating physical IoT sensors with a robust Java Spring Boot backend and an AI prediction engine. The primary objectives are:

- Real-time telemetry ingestion from underwater IoT sensor nodes over MQTT and REST APIs.
- Predictive analysis of oxygen depletion and biomass growth using machine learning.
- Automated actuation of surface aerators and automated feeders based on safety thresholds.
- Instant alert dispatching via SMS and Email to prevent crop losses.

2. Software Process Model & Schedule

2.1 Process Model Selection

The project adopts the Agile-Iterative Process Model. This methodology is selected due to the dual software-hardware nature of IoT projects, allowing rapid feedback loops, iterative feature development, and incremental integration testing.

2.2 Milestones & Deliverables

M1: Inception & Architecture Baseline (Weeks 1-3): Finalize system SRS, complete UML modeling, set up PostgreSQL schema, and initialize Spring Boot project.

M2: IoT Data Pipeline Implementation (Weeks 4-6): Deploy simulated sensor nodes, implement MQTT subscriber endpoints, and build real-time logging.

M3: AI Prediction Engine & Actuation Logic (Weeks 7-9): Train DO prediction models, build auto-threshold evaluation algorithms, and integrate SMS notification APIs.

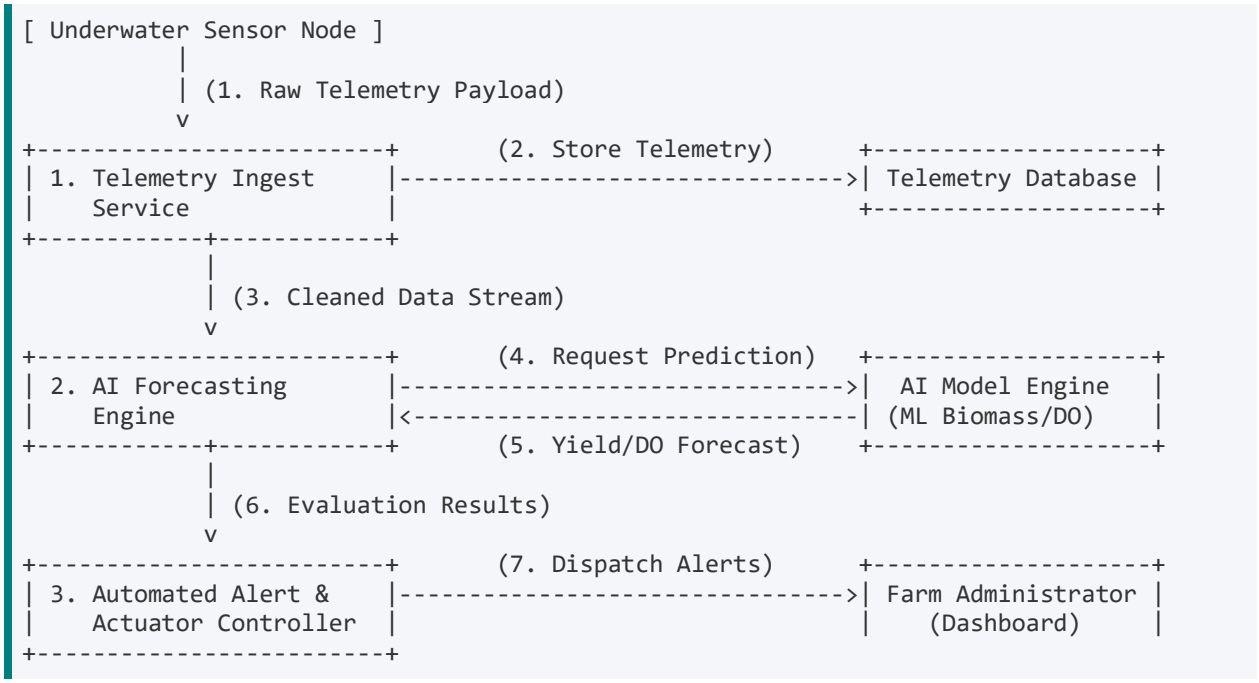
M4: Web UI Dashboard & System Deployment (Weeks 10-12): Develop Thymeleaf/Bootstrap UI, conduct end-to-end load testing, and deploy to production cloud server.

2.3 Project Schedule & Gantt Chart Matrix

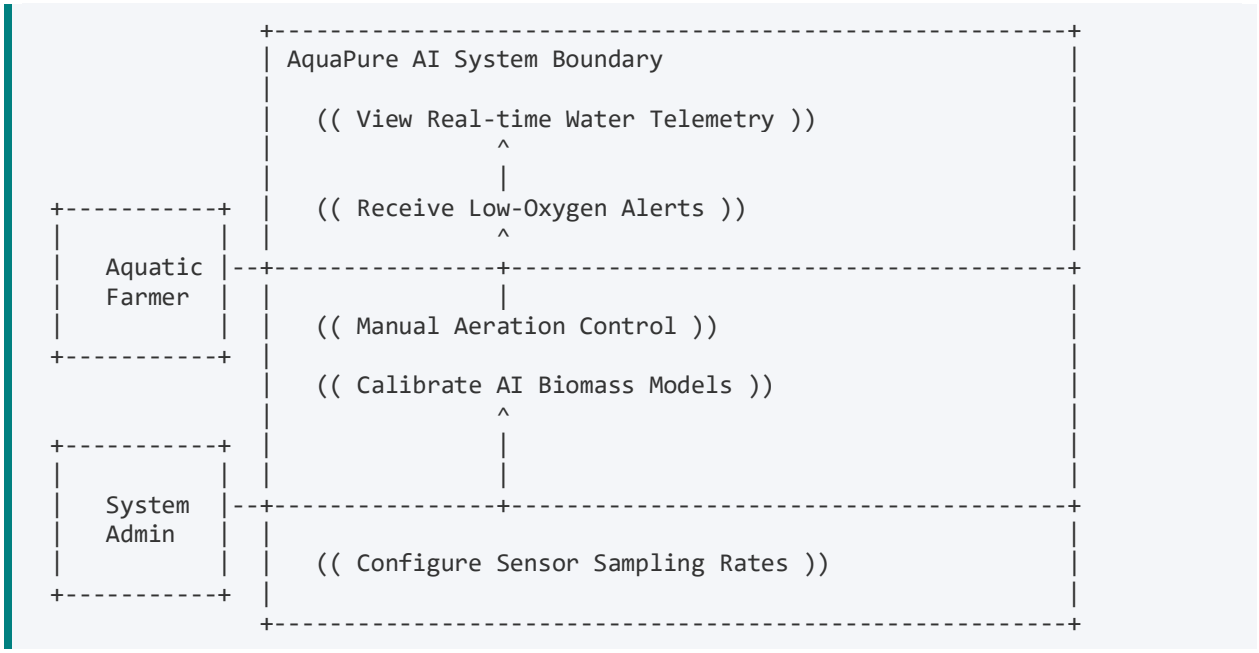
Task ID	Phase / Deliverable	W1-2	W3-4	W5-6	W7-8	W9-10	W11-12
T1	Requirements & Architecture UML	■■■					
T2	Spring Boot Backend Setup		■■■				
T3	IoT Telemetry Pipeline & DB			■■■			
T4	AI Dissolved Oxygen Model				■■■		
T5	Actuator Relay Control					■■■	
T6	Dashboard UI & Cloud Deployment						■■■

3. System UML Diagrams

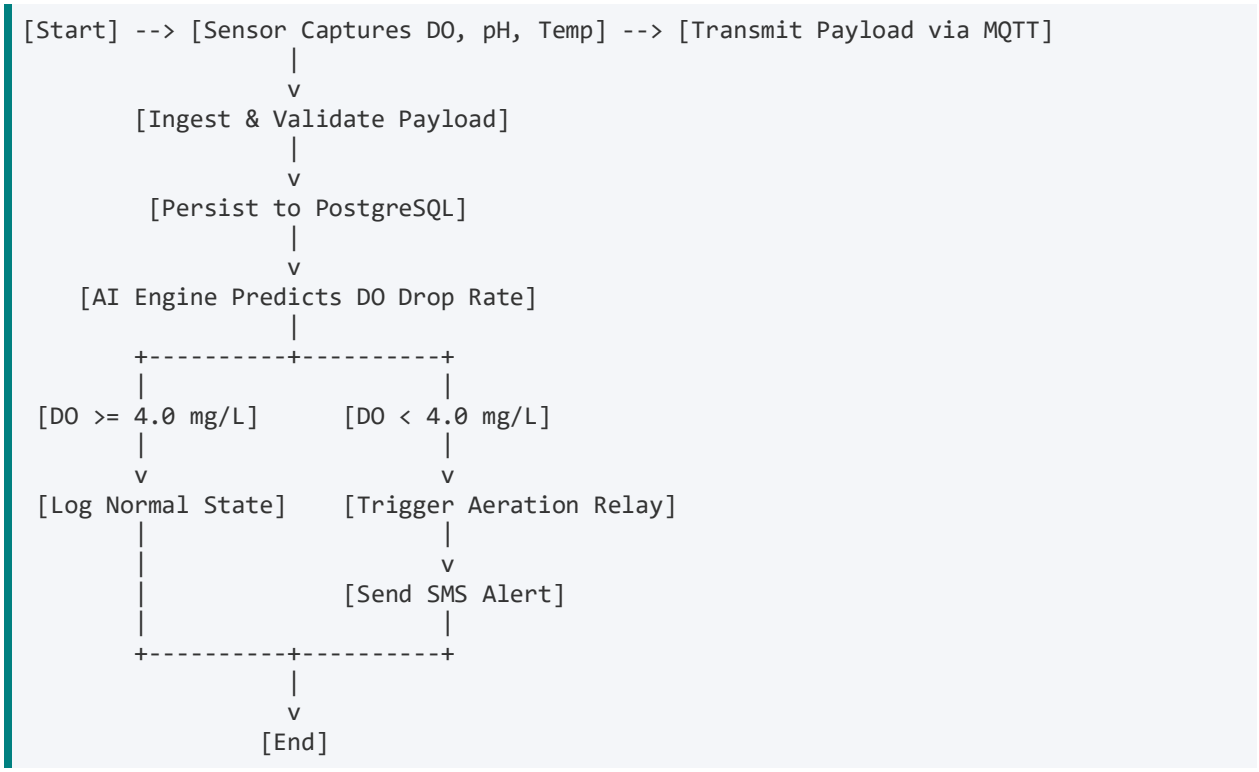
3.1 Data Flow Diagram (DFD Level 1)



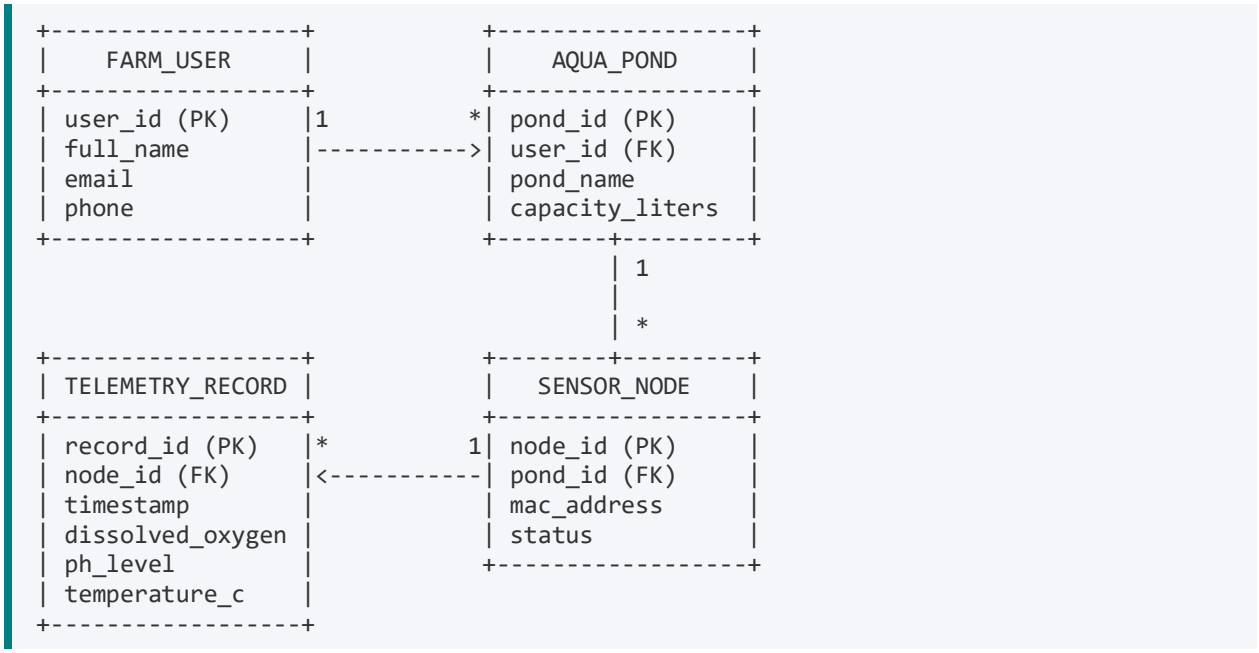
3.2 Use Case Diagram



3.3 Activity Diagram



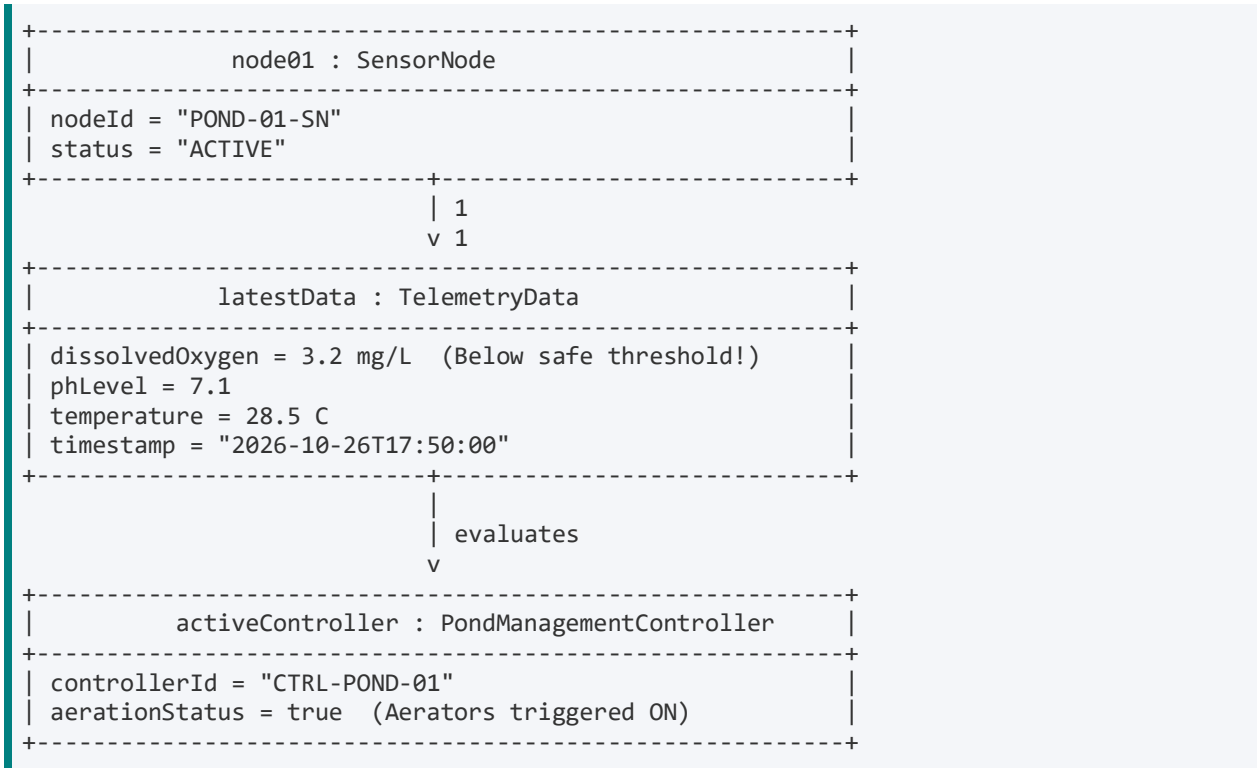
3.4 Entity-Relationship Diagram (ERD)



3.5 Class Diagram



3.6 Object Diagram (Runtime Instance State)



4. Java Software Implementation

Below is the complete Java object-oriented implementation demonstrating the interaction between the 5 core classes derived from the Class Diagram:

```
package com.aquapure.ai;

import java.time.LocalDateTime;

// 1. Telemetry Data Model Class
class TelemetryData {
    private double dissolvedOxygen; // mg/L
    private double phLevel;
    private double temperature;     // Celsius
    private LocalDateTime timestamp;

    public TelemetryData(double dissolvedOxygen, double phLevel, double temperature) {
        this.dissolvedOxygen = dissolvedOxygen;
        this.phLevel = phLevel;
        this.temperature = temperature;
        this.timestamp = LocalDateTime.now();
    }

    public double getDissolvedOxygen() { return dissolvedOxygen; }
    public double getPhLevel() { return phLevel; }
    public double getTemperature() { return temperature; }
    public LocalDateTime getTimestamp() { return timestamp; }
```

```
    public boolean isCriticalDO() {
        return this.dissolvedOxygen < 4.0; // Critical threshold for aquatic life
    }
}

// 2. Physical Sensor Node Representation
class SensorNode {
    private String nodeId;
    private String status;

    public SensorNode(String nodeId) {
        this.nodeId = nodeId;
        this.status = "ACTIVE";
    }

    public TelemetryData captureCurrentReadings(double doVal, double ph, double temp) {
        System.out.println("[" + nodeId + "] Capturing water telemetry...");
        return new TelemetryData(doVal, ph, temp);
    }

    public String getNodeId() { return nodeId; }
    public String getStatus() { return status; }
}

// 3. AI Prediction Engine
class AIPredictionEngine {
    private String modelVersion = "v2.1-Aquaculture";

    public double forecastOxygenDrop(TelemetryData data) {
        // High water temp accelerates DO consumption rate
        double dropRate = (data.getTemperature() > 27.0) ? 1.5 : 0.6;
        return data.getDissolvedOxygen() - dropRate;
    }
}

// 4. Alert Notification Dispatcher
class AlertNotification {
    private String alertId;
    private String severity;
    private String message;

    public AlertNotification(String alertId, String severity, String message) {
        this.alertId = alertId;
        this.severity = severity;
        this.message = message;
    }

    public void dispatchAlert(String recipientEmail) {
        System.out.println(">>> ALERT DISPATCHED [" + severity + "] to " +
recipientEmail);
        System.out.println("    Message: " + message);
    }
}

// 5. Central System Controller Interface
public class PondManagementController {
    private String controllerId;
```

```
    private boolean aerationStatus;
    private AIPredictionEngine aiEngine;

    public PondManagementController(String controllerId) {
        this.controllerId = controllerId;
        this.aerationStatus = false;
        this.aiEngine = new AIPredictionEngine();
    }

    public void processTelemetryPayload(SensorNode node, TelemetryData data) {
        System.out.println("Processing payload from Node: " + node.getNodeId());
        System.out.println("DO: " + data.getDissolvedOxygen() + " mg/L | pH: " +
data.getPhLevel() + " | Temp: " + data.getTemperature() + " C");

        double predictedDO = aiEngine.forecastOxygenDrop(data);
        System.out.println("AI Forecasted DO in 4 Hours: " + predictedDO + " mg/L");

        if (data.isCriticalDO() || predictedDO < 3.5) {
            this.aerationStatus = true;
            System.out.println("Automated Relay Trigger: Surface Aerator SWITCHED
ON.");

            AlertNotification alert = new AlertNotification(
                "ALT-" + System.currentTimeMillis(),
                "CRITICAL",
                "Low Dissolved Oxygen detected (" + data.getDissolvedOxygen() + "
mg/L). Aerator activated automatically."
            );
            alert.dispatchAlert("aquaculture_manager@farm.com");
        } else {
            System.out.println("Water parameters within nominal safe ranges.");
        }
    }

    public static void main(String[] args) {
        System.out.println("=== AquaPure AI Controller Test ===");
        PondManagementController controller = new PondManagementController("CTRL-POND-
01");
        SensorNode pondSensor = new SensorNode("POND-01-SN");

        // Simulate incoming low-oxygen telemetry data
        TelemetryData criticalReading = pondSensor.captureCurrentReadings(3.2, 7.1,
28.5);
        controller.processTelemetryPayload(pondSensor, criticalReading);
    }
}
```

5. Submission Details

GitHub Repository Link: https://github.com/siamjshoishob1925-cloud/-Software-Engineering-And-Project-Management/tree/main/Project